Azure AI Content Safety & Responsible-AI Guardrails

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — Azure AI Content Safety & Responsible-AI Guardrails 5

What this is (and is not) . . . 5

What you get . . . 5

The deployment model . . . 6

The boundary . . . 6

Effort . . . 6

Outcome . . . 6

1. Solution Overview — Azure AI Content Safety & Responsible-AI Guardrails 6

What Azure AI Content Safety is . . . 6

How it fits into your app (the filtering pattern) . . . 7

The severity model (what you actually tune) . . . 7

Auth and connections used . . . 8

The standard deployment model . . . 8

Scope . . . 8

Out of scope . . . 8

Benefits . . . 8

Limitations (be honest with buyers) . . . 9

2. Requirements Guide — Azure AI Content Safety & Responsible-AI Guardrails 9

2.1 Azure AI Content Safety resource availability . . . 9

2.2 A custom AI app in scope (with an owner) . . . 10

2.3 Keyless authentication (managed identity + RBAC) . . . 10

2.4 Python toolchain and the Content Safety SDK . . . 10

2.5 Severity policy (thresholds ratified by trust & safety) . . . 10

2.6 Custom blocklist source (if used) . . . 10

2.7 Grounding sources for groundedness detection (RAG apps) . . . 11

2.8 Observe-before-enforce and rollback (change control) . . . 11

2.9 Responsible-AI review, red-team, and incident/appeal process owners . . . 11

Checklist . . . 11

Global Requirements — AI (category-level) 12

G1. Azure subscription with AI service access and rights . . . 12

G2. Entra identity, managed identity, and RBAC . . . 12

G3. Private networking capability . . . 12

G4. IaC + Python tooling and source control . . . 12

G5. CI/CD platform . . . 13

G6. Data source and classification (for data-grounded solutions) . . . 13

G7. Content-safety and responsible-AI posture . . . 13
G8. Cost controls and quotas . . . 13
3. Implementation Guide — Azure AI Content Safety & Responsible-AI Guardrails 13
Step 1 — Assess the AI apps and current guardrails (read-only) . . . 14
Step 2 — Provision the Content Safety resource (keyless) . . . 14
Step 3 — Grant the app keyless access (managed identity + RBAC) . . . 15
Step 4 — Define the severity policy (trust & safety ratifies) . . . 15
Step 5 — Create the custom blocklist (safe preview first) . . . 15
Step 6 — Build the safety filter (pre/post-generation), observe mode . . . 16
Step 7 — Integrate on the pilot app and run in observe mode . . . 16
Step 8 — Stand up the responsible-AI review . . . 16
Step 9 — Stand up the incident and appeal process . . . 17
Step 10 — Validate against the gate, then flip to enforce (pilot pass) . . . 17
Step 11 — Expand to more apps . . . 17
5. Testing Guide — Azure AI Content Safety & Responsible-AI Guardrails 17
Test 1 — Guardrail configuration is correctly wired . . . 18
Test 2 — Text moderation catches harmful input to policy . . . 18
Test 3 — Benign input is NOT over-blocked (false-positive check) . . . 18
Test 4 — Custom blocklist flags its terms . . . 18
Test 5 — Prompt Shields catch jailbreak and indirect injection . . . 18
Test 6 — Post-generation moderation catches harmful output . . . 18
Test 7 — Groundedness detection flags hallucinations (RAG apps) . . . 19
Test 8 — Protected-material detection (where available) . . . 19
Test 9 — Observe → enforce behaves correctly . . . 19
Test 10 — Incident and appeal path works . . . 19
Pass/fail gate . . . 19
6. Troubleshooting Guide — Azure AI Content Safety & Responsible-AI Guardrails 19
A. Authentication and connectivity . . . 19
B. Over-blocking (false positives) . . . 20
C. Under-blocking (harmful content gets through) . . . 20
D. Prompt Shields / indirect injection . . . 21
E. Groundedness detection issues . . . 21
F. Protected-material detection . . . 21
G. Latency and cost . . . 21
H. Diagnostics quick reference . . . 22
I. When to escalate . . . 22

7. Best Practices — Azure AI Content Safety & Responsible-AI Guardrails 22
Where to go next 23
8. Deliverables Summary — Azure AI Content Safety & Responsible-AI Guardrails 24
Documentation 24
Python scripts 24
What the buyer receives 25
Outcome 25
Appendix A — PowerShell Scripts 26

Executive Summary — Azure AI Content Safety & Responsible-AI Guardrails

A custom AI application — a chatbot, a copilot, a document assistant — is only as safe as the guardrails around it. Ship one with no safety layer and it can emit hateful or violent content, be talked out of its own rules by a jailbreak prompt, follow instructions hidden in a document it was asked to summarize, hallucinate a confident answer that has no basis in your data, or reproduce copyrighted text. Each of those is a brand, legal, and compliance risk — and none of them is caught by the language model itself. They are caught by a **guardrail layer** you put in front of and behind generation.

This is a complete, deploy-it-in-order package for adding that guardrail layer to your **own** custom AI apps using **Azure AI Content Safety**, written for a competent developer or administrator who has not built AI safety controls before. It gives you the repeatable way to moderate text and images against harm categories with severity thresholds, shield your prompts against jailbreak and indirect prompt-injection attacks, enforce a custom blocklist, check answers for groundedness and protected material, wire all of it into a **pre-generation / post-generation filter**, and stand up a responsible-AI review, red-team, and incident/appeal process — proven on a pilot app before you put it in front of users.

What this is (and is not)

This solution is content-safety **guardrails for custom AI apps you build** — apps that call Azure OpenAI or another model directly. It is deliberately **distinct from AI governance for Microsoft 365 Copilot** (which is about permissions, sensitivity labels, and DLP over your tenant data — see the *Microsoft 365 Copilot Secure Adoption & Governance* and *Microsoft Purview* solutions). If your risk is "Copilot can see too much," that is the other solution. If your risk is "the app I built could say or be tricked into something harmful," this is the one.

What you get

- A plain-language Solution Overview of how Azure AI Content Safety works — the four harm categories

and severity scale, Prompt Shields, blocklists, groundedness detection, protected-material detection — and the pre/post-generation filtering pattern that ties them into your app.

- A Requirements Guide with verify / implement-if-missing for every prerequisite specific to content

safety (the category-wide Azure/AI prerequisites are published once in the global requirements).

- An Implementation Guide of precise, ordered steps — provision the Content Safety resource keyless,

define a severity policy and blocklist, build the safety filter, add Prompt Shields and groundedness checks, then pilot on one app and validate with red-team prompts before expanding.

- Five Python scripts (using the `azure-ai-contentsafety` SDK) for discovery, configuration

validation, a moderation-posture report, a ready-to-adapt pre/post-generation **safety filter** integration sample, and a safe blocklist manager.

- Testing and Troubleshooting guides with explicit pass/fail gates, plus a responsible-AI review and

incident/appeal process.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — pilot the guardrails on one app, in audit/observe mode first, with a rollback at every stage before you enforce blocking in production.

The boundary

This guide delivers the **technical controls and the process scaffolding**. What content is "acceptable," where each severity threshold should sit, and what your responsible-AI and acceptable-use policy says are **the customer's legal / trust-and-safety / compliance decisions** — the guide shows you how to enforce those decisions once they are made, not what they should be.

Effort

A competent developer/admin can stand up the Content Safety resource and a working safety filter on a pilot app in about a day, then spend a few days red-teaming, tuning thresholds with trust & safety, and wiring the incident process before expanding to more apps. No consultant required.

Outcome

Custom AI apps with real guardrails a single team can run and defend at review: harmful content moderated to a defined policy, jailbreak and injection attacks shielded, answers checked for groundedness and protected material, and a responsible-AI review, red-team, and incident/appeal process behind it — validated on a pilot before any app ships.

1. Solution Overview — Azure AI Content Safety & Responsible-AI Guardrails

What Azure AI Content Safety is

Azure AI Content Safety is a cloud service that inspects **text and images** — both the input going into your AI app and the output coming out of it — and returns a structured safety judgment your code can act on. It does not replace the language model; it sits **around** the model as a guardrail. Your app calls Content Safety before it sends a prompt to the model (to catch harmful or adversarial input) and after the model responds (to catch harmful or ungrounded output) — the **pre-generation / post-generation filter** pattern this solution builds.

It exposes several capabilities through one resource and SDK:

- **Text moderation** — `analyze_text` scores content across four harm categories: **Hate**,

Sexual, **Violence**, and **Self-Harm**. Each category comes back with a **severity** level (a 0–7 scale, or a coarser 0/2/4/6 four-level mode). Your app compares the severity to a threshold *you* set per category and decides to allow, block, or route for human review.

- **Image moderation** — `analyze_image` scores images across the same four categories with severity,

for apps that accept or generate images.

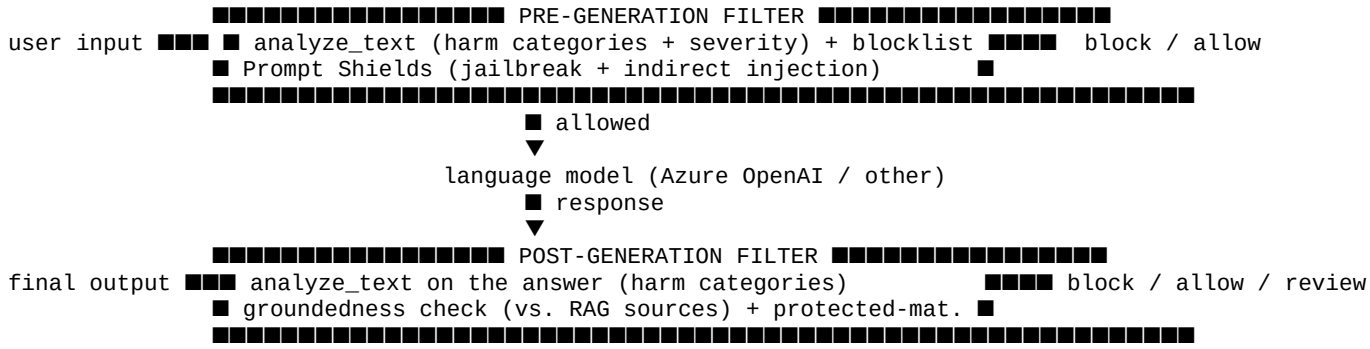
• **Custom blocklists** — term lists you define (`analyze_text` with `blocklist_names`) so specific words or phrases are always flagged regardless of the category model — useful for brand-specific, domain-specific, or competitor/PII terms.

• **Prompt Shields** — a detector for **jailbreak** attacks (user attempts to override the app's instructions, e.g. "ignore your rules and…") and **indirect prompt-injection** attacks (malicious instructions hidden inside a document, web page, or email the app was asked to process). *Prompt Shields, and the exact API shape, evolve — verify current availability and API version in your region.*

• **Groundedness detection** — checks whether a model's answer is actually **supported by the source material** you grounded it on (your RAG documents), flagging confident hallucinations. *Preview capability — verify current availability in your tenant/region.*

• **Protected-material detection** — flags output that reproduces known **copyrighted text** (and, in preview, code). *Verify current availability.*

How it fits into your app (the filtering pattern)



1. **Pre-generation:** run the user's input (and any retrieved documents) through text moderation, the blocklist, and Prompt Shields. If input exceeds your severity policy or trips a shield, block it or sanitize it **before** it ever reaches the model.
1. **Generation:** the model produces a candidate answer.
2. **Post-generation:** run the candidate answer through moderation again (the model can produce harm even from clean input), check it for groundedness against the sources, and check for protected material. Block, redact, or route to human review per policy before returning it to the user.
1. **Log every decision** (categories, severities, shield hits, groundedness) for the responsible-AI review and the incident/appeal process.

The severity model (what you actually tune)

Content Safety returns a severity per category; **you** own the threshold. A typical starting policy is "block at severity ≥ 4 (medium/high) for all four categories, flag 2 for review, allow 0" — but the correct thresholds depend on your audience and use case and are a **trust-and-safety decision**, not a technical default. This solution gives you the mechanism and a safe starting point; the policy numbers are ratified by the customer.

Auth and connections used

• **Azure AI Content Safety resource** (an Azure AI Services / Cognitive Services resource) — the endpoint your app calls.

• **Keyless authentication by default** — the app authenticates with a **managed identity** assigned the **Cognitive Services User** role on the resource (`DefaultAzureCredential`), so no API key is stored in the app. A key is supported only as a fallback and, if used, lives in **Key Vault**, never in code (see global requirements G2).

• **Your existing AI app** and, for groundedness, your **RAG source documents**.

• **Private networking** where required (global requirement G3) — the Content Safety resource can be reached over a private endpoint with public access disabled.

The standard deployment model

> ***Assess*** *which of your AI apps generate or accept content and have no guardrails today →* ***Configure*** *the Content Safety resource (keyless), the severity policy, and the blocklist →* ***Pilot*** *the safety filter on one app in* ***audit/observe mode*** *(log, don't block) →* ***Validate*** *with red-team prompts and groundedness tests against a pass/fail gate →* ***Expand*** *to enforce blocking and roll the pattern out to more apps. The pattern is* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand****, with observe-before-enforce and a rollback at every stage.*

Scope

• Adding Azure AI Content Safety guardrails (text/image moderation, blocklists, Prompt Shields, groundedness, protected-material) to **custom AI apps you own**.

• Building the reusable pre/post-generation **safety filter** integration and its logging.

• Standing up the responsible-AI review, red-team prompt set, and incident/appeal process.

Out of scope

• **Microsoft 365 Copilot / Purview AI governance** — permissions, sensitivity labels, and DLP over tenant data are a *different* problem; use the Copilot and Purview solutions (cross-referenced, not duplicated here).

• **Building the AI app itself, the model deployment, or the RAG pipeline** — this solution guardrails an app you already have (see the AI landing-zone / RAG solutions for building it).

• **Deciding what content is acceptable or setting the responsible-AI policy** — those determinations belong to the customer's legal / trust-and-safety / compliance functions; this guide enforces them.

Benefits

• **Harm moderated to policy** — text and images scored against four categories with thresholds you own.

• **Attacks shielded** — jailbreak and indirect prompt-injection detected before they reach the model.

• **Hallucinations caught** — groundedness checks flag answers unsupported by your sources.

• **Brand & IP protected** — blocklists and protected-material detection catch specific terms and copyrighted output.

• **Defensible process** — a responsible-AI review, red-team set, and incident/appeal process behind the controls, all logged and repeatable.

Limitations (be honest with buyers)

• Content Safety is **probabilistic** — it reduces risk, it does not guarantee zero harmful output; keep human review in the loop for high-stakes uses.

• **Groundedness and protected-material detection are preview/evolving** capabilities — confirm current availability, region support, and API versions in your tenant before you depend on them.

• Prompt Shields catch known attack patterns; **novel jailbreaks will appear** — red-teaming is ongoing, not one-time.

• **Latency and cost** — every pre/post call adds a round trip and a per-transaction charge; budget and quota accordingly (global requirement G8).

• **Thresholds are a policy call** — the service gives you severities; where to draw the line is the customer's trust-and-safety decision.

2. Requirements Guide — Azure AI Content Safety & Responsible-AI Guardrails

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (Azure subscription and rights, Entra + managed identity, private networking, Python/IaC tooling, CI/CD, data source, responsible-AI posture, cost controls) are published once in `global-requirements-ai.md` — they are **referenced, not repeated** here. The items below are specific to Azure AI Content Safety.

2.1 Azure AI Content Safety resource availability

Description: Azure AI Content Safety available in a region you can deploy to (a standalone Content Safety resource, or the equivalent capability on an Azure AI Services multi-service resource). **Why:** The resource is the endpoint your app's safety filter calls; region and capability availability vary. **Verify:** In the Azure portal, **Create a resource** → **search "Content Safety"** resolves and offers a region near your app. Or `az cognitiveservices account list-kinds` shows the Content Safety kind. **Implement:** Choose a supported region (co-locate with your AI app to minimize latency); confirm your subscription can

create the resource. *AI service regions/features move — verify current availability.*

2.2 A custom AI app in scope (with an owner)

Description: At least one AI app you own — chatbot, copilot, document assistant — that generates or accepts text/images, identified as the pilot, with a named app owner. **Why:** This solution guardrails an app you already run; the pilot app is where the safety filter is first integrated and validated. **Verify:** Run `AISAFETY-Discovery.py` to inventory in-scope AI endpoints and whether any safety layer exists today; confirm the app owner. **Implement:** Pick one representative app as the pilot; if you have no app yet, build it first (see the AI landing-zone / RAG solutions) — guardrails wrap an existing app.

2.3 Keyless authentication (managed identity + RBAC)

Description: A managed identity for the app (or the developer's Azure identity for local testing) holding the **Cognitive Services User** role on the Content Safety resource. **Why:** Keys leak; the app should authenticate to Content Safety via managed identity + RBAC with no key in code (global requirement G2). **Verify:** In the resource → **Access control (IAM)**, the app's managed identity has *Cognitive Services User*. `DefaultAzureCredential` succeeds from the app context. **Implement:** Enable a managed identity on the app compute; assign **Cognitive Services User** at the resource scope; use `DefaultAzureCredential`. If a key is genuinely unavoidable, store it in **Key Vault**, never in source.

2.4 Python toolchain and the Content Safety SDK

Description: Python 3.8+ with `azure-ai-contentsafety` and `azure-identity` installed (plus `requests` for preview REST endpoints), pinned in the project. **Why:** The scripts and the safety-filter integration are Python using the Content Safety SDK. **Verify:** `python --version` ≥ 3.8; `pip show azure-ai-contentsafety azure-identity` returns versions. *The AI SDKs move fast — verify current package versions and pin them (global G4).* **Implement:** `pip install azure-ai-contentsafety azure-identity requests`; pin versions in `requirements.txt`; store in Git.

2.5 Severity policy (thresholds ratified by trust & safety)

Description: A documented content-severity policy: per-category (Hate / Sexual / Violence / Self-Harm) block and review thresholds on the 0–7 scale, named `Content Severity Policy – App Guardrails – Pilot`. **Why:** The service returns severities; **where to draw the block line is a trust-and-safety policy decision**, not a technical default — it must be owned and signed off. **Verify:** A written policy exists with per-category thresholds and an owner. The safety filter reads these thresholds from config, not hard-coded values. **Implement:** Draft a starting policy (e.g. block ≥ 4, review = 2, allow 0) and have the customer's **legal / trust-and-safety** function ratify it before enforcing. *Acceptable-content determinations are the customer's decision (global G7).*

2.6 Custom blocklist source (if used)

Description: A list of terms/phrases to always flag — brand names, banned terms, competitor or PII patterns — for the blocklist `Prohibited Terms – App Guardrails – Pilot`. **Why:** The category model catches general harm; a blocklist enforces organization-specific terms the model would not know about. **Verify:** A reviewed term list exists (owned by trust & safety / legal, not invented by the

implementer). **Implement:** Collect and review the term list; load it with `AISAFETY-Blocklist.py` (which defaults to a safe preview). Keep the list in source control (excluding anything sensitive).

2.7 Grounding sources for groundedness detection (RAG apps)

Description: For apps that ground answers on your data (RAG), the **source passages** the app retrieved for a given answer, available to pass to the groundedness check. **Why:** Groundedness detection compares the answer to its sources; it needs the sources as input. **Verify:** The app can surface the retrieved context for each answer (it usually already has it). **Implement:** Wire the retrieved passages through to the post-generation filter. *Groundedness detection is a preview capability — verify current availability and API version in your region.*

2.8 Observe-before-enforce and rollback (change control)

Description: A documented plan to run the safety filter in **audit/observe mode** (log decisions, do not block) on the pilot first, with a defined rollback. **Why:** Blocking user content is disruptive; false positives must be measured before enforcement, and a rollback is mandatory (global G8, and the pilot-before-enforce house rule). **Verify:** The filter has an `observe` mode flag; a change ticket names the pilot app, an approver, and the rollback (revert to observe / bypass the filter). **Implement:** Default the filter to observe mode; measure false-positive rate on the pilot; only flip to enforce after the trust-and-safety gate passes.

2.9 Responsible-AI review, red-team, and incident/appeal process owners

Description: Named owners for the responsible-AI review, the red-team prompt set (`Red-Team Prompt Set – App Guardrails – Pilot`), and the incident/appeal process (`RAI Incident & Appeal Process – App Guardrails`). **Why:** Guardrails are a process, not a one-time config — someone must review flagged content, run red-team cycles, and handle user appeals of blocked content. **Verify:** Owners are named and the process documents exist (this package supplies the templates). **Implement:** Assign owners; adopt the review/red-team/incident templates in Implementation Steps 7–9. *Responsible-AI policy determinations are the customer's legal / trust-and-safety decision (global G7).*

Checklist

- Azure AI Content Safety resource available in a supported region
- Pilot AI app identified with a named owner (`AISAFETY-Discovery.py`)
- Keyless auth: app managed identity has *Cognitive Services User* on the resource
- Python 3.8+ with `azure-ai-contentsafety` / `azure-identity` pinned
- `Content Severity Policy – App Guardrails – Pilot` drafted and ratified by trust & safety
- Blocklist term source reviewed (if using `Prohibited Terms – App Guardrails – Pilot`)
- Grounding sources available to the post-generation filter (RAG apps)
- Safety filter defaults to observe mode; rollback documented
- RAI review / red-team / incident-and-appeal owners named

Global Requirements — AI (category-level)

These requirements apply to every solution in the AI category (building and operating custom AI on Azure — Azure OpenAI / AI Foundry, RAG, Azure ML / MLOps, AI landing zones, content safety). Published once at the category level so listings stay small. Confirm before deploying any AI solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Custom AI touches sensitive data and can emit harmful or ungrounded output. Deploy private-by-default (no public endpoints, keyless auth), ground and permission-trim on your own data, put safety guardrails in front of generation, and pilot in non-production before production — content-acceptability and AI-use-policy decisions belong to the customer's legal / trust-and-safety / compliance functions, not this guide.*

G1. Azure subscription with AI service access and rights

Description: An Azure subscription with access to the required AI services (Azure OpenAI / AI Foundry, Azure AI Search, Azure Machine Learning, Azure AI Content Safety) and Contributor/Owner at the deployment scope. Why: These solutions create and configure real, billable Azure AI resources. Verify: `az account show` succeeds; the AI resource providers are registered and the region is available. *Azure OpenAI/model access and regions vary — verify current availability in your subscription.* Implement: Obtain the subscription/role; request access to the gated AI services where required; pick a supported region.

G2. Entra identity, managed identity, and RBAC

Description: Entra ID identities plus managed identities for keyless service-to-service auth. Why: Keys leak; AI services should authenticate via managed identity + RBAC (least privilege). Verify: Managed identities can be assigned and the needed data-plane roles exist (e.g. Cognitive Services OpenAI User, Search Index Data Reader). Implement: Enable managed identity on the compute/app; assign least-privilege data-plane roles; store any unavoidable secret in Key Vault.

G3. Private networking capability

Description: A VNet with the ability to create private endpoints and private DNS for the AI services. Why: Public endpoints are a data-exfiltration surface; enterprise AI is private-by-default. Verify: The VNet/subnets and Private DNS zones exist or can be created; public access can be disabled. Implement: Provision the VNet, subnets, and Private DNS; plan private endpoints per service (see the AI landing-zone solution).

G4. IaC + Python tooling and source control

Description: Terraform/Bicep, the Azure CLI, Python 3.x with the relevant Azure AI SDKs, and Git. Why: AI platforms are delivered as versioned code (infra + pipelines + app), so they are reproducible and reviewable. Verify: `az version`, `terraform version`/`az bicep version`, `python --version`, and `git --version` return supported builds. *Verify current SDK/provider versions — the AI SDKs move fast.* Implement: Install and pin the toolchain; store all config in Git.

G5. CI/CD platform

Description: A pipeline (GitHub Actions / Azure DevOps) with OIDC/workload-identity federation to Azure. Why: Infra, model, and app deployments must run through an auditable, approval-gated pipeline — not a laptop. Verify: The pipeline can authenticate to Azure via OIDC and run a plan/deploy step. Implement: Configure the service connection with workload-identity federation; scope its rights.

G6. Data source and classification (for data-grounded solutions)

Description: The private data to ground on (documents, databases) with a known permission/ACL model, and a sensitivity-classification position. Why: RAG must respect source permissions (permission-trimming) and must not expose sensitive data through AI. Verify: The data source, its access model, and applicable sensitivity labels are known. Run the solution's discovery script. Implement: Inventory the data and its ACLs; align sensitivity labeling (see the Purview solutions). *Data-classification determinations are the compliance team's decision.*

G7. Content-safety and responsible-AI posture

Description: An organizational position on acceptable AI content, safety thresholds, and responsible-AI review. Why: Generative AI can emit harmful/ungrounded output and be jailbroken; the organization must define guardrails and review. Verify: A responsible-AI / acceptable-use policy exists; content-safety is enabled where generation occurs. Implement: Adopt an RAI/acceptable-use policy and enable content safety (see the content-safety solution). *Acceptable-content and RAI determinations are the customer's legal / trust-and-safety decision.*

G8. Cost controls and quotas

Description: Budgets, alerts, and per-service quotas (e.g. Azure OpenAI TPM) appropriate to the workload. Why: AI services bill on tokens/compute; unmanaged usage causes bill shock. Verify: Budgets/alerts exist and quotas are set for the deployed services. Implement: Set budgets, cost alerts, and quotas/rate limits before broad rollout. *Spend limits are the customer's business decision.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Azure AI Content Safety & Responsible-AI Guardrails

Follow these steps **in order**, across the **Azure portal** (`https://portal.azure.com`), your **AI app's code**, and the supplied Python scripts. Where a portal action has a CLI equivalent it is shown so the work is reproducible.

> ***Model:*** *Provision the guardrail resource keyless, define the severity policy and blocklist, build the safety filter, then integrate it into **one** pilot app in **observe mode** (log, don't block), validate with red-team prompts, and only then flip to enforce and expand to more apps. Observe before enforce; roll back to observe at any sign of over-blocking.*

Name every object exactly as shown, in the form `<Purpose> – <Scope> – <Stage>`, and reuse the names verbatim across scripts, config, and reports:

Object	Name
Resource group	`rg-ai-guardrails-pilot`
Content Safety resource	`AI Content Safety – App Guardrails – Pilot`
Severity policy (config doc)	`Content Severity Policy – App Guardrails – Pilot`
Custom blocklist	`Prohibited Terms – App Guardrails – Pilot`
Safety filter component	`Safety Filter – Pre-Post Generation – Pilot`
Red-team prompt set	`Red-Team Prompt Set – App Guardrails – Pilot`
Incident & appeal process	`RAI Incident & Appeal Process – App Guardrails`

Step 1 — Assess the AI apps and current guardrails (read-only)

1. Run `AISAFETY-Discovery.py` to inventory the in-scope AI apps/endpoints, whether an Azure AI Content

Safety resource already exists in the subscription, and whether each app calls any safety layer today (pre/post-generation).

1. Record, per app: does it accept user text/images? does it generate content? does it ground answers on

your data (RAG)? is there any moderation today? Pick **one** representative app as the pilot and confirm its owner.

1. Do **not** change any app yet. The output of this step is the worklist and the chosen pilot app.

Step 2 — Provision the Content Safety resource (keyless)

1. In the Azure portal, **Create a resource** → **"Content Safety"** → **Create**. Set:

• **Resource group:** `rg-ai-guardrails-pilot`

• **Name:** `AI Content Safety – App Guardrails – Pilot` (Azure resource names have character

limits — if the display name is rejected, use `ai-contentsafety-guardrails-pilot` as the resource name and keep the friendly name in your config/docs).

• **Region:** co-located with the pilot app; **Pricing tier:** per your expected volume.

CLI equivalent:

```
az group create -n rg-ai-guardrails-pilot -l <region>
az cognitiveservices account create \
  -n ai-contentsafety-guardrails-pilot -g rg-ai-guardrails-pilot \
  --kind ContentSafety --sku S0 -l <region> --custom-domain ai-contentsafety-guardrails-pilot
```

1. **Disable public network access** where your architecture requires it and add a **private endpoint**

(global requirement G3): resource → **Networking** → **Private access**. Reach it from the app's VNet.

1. Turn on **diagnostic settings** → send logs to your Log Analytics workspace so every moderation call is auditable (resource → **Monitoring** → **Diagnostic settings** → **Add**).

Step 3 — Grant the app keyless access (managed identity + RBAC)

1. Enable a **system-assigned managed identity** on the pilot app's compute (App Service / Container App / VM / AKS workload identity): app → **Identity** → **System assigned** → **On**.

1. On the Content Safety resource → **Access control (IAM)** → **Add role assignment**, assign **Cognitive Services User** to the app's managed identity (and to your own dev identity for local testing).

```
az role assignment create --assignee <app-managed-identity-object-id> \
  --role "Cognitive Services User" \
  --scope /subscriptions/<sub>/resourceGroups/rg-ai-guardrails-pilot/providers/Microsoft.CognitiveServi
```

1. Confirm the app can authenticate with `DefaultAzureCredential` — **no key in code**. If a key is genuinely unavoidable, store it in **Key Vault** and reference it; never hard-code it.

1. Set the endpoint as app configuration, e.g. `CONTENT_SAFETY_ENDPOINT=https://<resource>.cognitiveservices.azure.com/`.

Step 4 — Define the severity policy (trust & safety ratifies)

1. Create the config document `Content Severity Policy – App Guardrails – Pilot` recording, per category (**Hate / Sexual / Violence / Self-Harm**), the **block** and **review** thresholds on the 0–7 scale. A safe starting point: **block** ≥ **4**, **review = 2**, **allow 0** for all four categories.

1. Have the customer's **legal / trust-and-safety** function **ratify** the thresholds — this is a policy decision, not a technical default. Record the owner and sign-off date.

1. Store the thresholds as **configuration** the safety filter reads (e.g. `severity_policy.json`), so changing policy never means changing code. The supplied scripts read this config.

Step 5 — Create the custom blocklist (safe preview first)

1. Assemble the reviewed term list (owned by trust & safety / legal). Run `AISAFETY-Blocklist.py --create --name "Prohibited Terms – App Guardrails – Pilot" --terms-file terms.txt` in its **default preview mode** to see exactly what it would create/add.

1. Re-run with `--apply` to create the blocklist `Prohibited Terms – App Guardrails – Pilot` and add the terms. The script is idempotent — re-running updates the list without duplicating it.

1. Note the blocklist name; the safety filter references it in every `analyze_text` call via

`blocklist_names`.

Step 6 — Build the safety filter (pre/post-generation), observe mode

1. Take `AISAFETY-SafetyFilter.py` as the starting point for the `Safety Filter – Pre-Post Generation –

Pilot` component. It implements:

• **Pre-generation:** `analyze_text` (four categories + severity) on user input **and** any retrieved

documents, the blocklist, and **Prompt Shields** (jailbreak + indirect injection). *Prompt Shields API shape evolves — verify the current API version for your region.*

• **Post-generation:** `analyze_text` on the model's answer, a **groundedness** check against the RAG

sources, and a **protected-material** check. *Groundedness and protected-material are preview/evolving — verify current availability.*

• A decision per stage: **allow / block / review**, driven by the `Content Severity Policy` config.

• Structured **logging** of every decision (categories, severities, shield hits, groundedness score)

for the RAI review and incident process.

1. Set the filter to `mode=observe` (default): it computes and logs a decision but does **not**

block — the app still returns its normal answer. This measures false positives without disrupting users.

1. Wire the filter into the pilot app: call the pre-filter before the model call, the post-filter after,

passing retrieved sources for groundedness.

Step 7 — Integrate on the pilot app and run in observe mode

1. Deploy the pilot app with the safety filter in observe mode. Confirm the app still works normally.

2. Let real (or scripted) traffic flow. Run `AISAFETY-Validate.py` to confirm the guardrail

configuration is correctly wired (resource reachable keyless, blocklist present, severity config loaded, Prompt Shields / groundedness endpoints reachable).

1. Run `AISAFETY-Report.py` to send the `Red-Team Prompt Set – App Guardrails – Pilot` and a set of

benign control prompts through the safety endpoints and produce a **moderation-posture report** — which prompts trip which categories/severities/shields, and the false-positive rate on benign inputs. (The red-team set contains *adversarial prompts to test the guardrails*; the report never emits harmful content, only the safety verdicts.)

Step 8 — Stand up the responsible-AI review

1. Establish a recurring **RAI review** of the logged decisions: sample blocked/flagged items, confirm

the thresholds are catching real harm and not over-blocking, and adjust the `Content Severity Policy` config (re-ratified by trust & safety) as needed.

1. Record the review cadence, participants, and where the logs live. This is the loop that keeps the

thresholds honest.

Step 9 — Stand up the incident and appeal process

1. Adopt the `RAI Incident & Appeal Process – App Guardrails`: how a user reports harmful output that

slipped through, how a user **appeals** content wrongly blocked, who triages, and the SLA.

1. Wire the appeal path into the app's UX (e.g. a "report / this was wrongly blocked" affordance) and

route it to the named owner from requirement 2.9.

Step 10 — Validate against the gate, then flip to enforce (pilot pass)

1. Work through the **Testing Guide**. The pilot gate is met when: harmful/adversarial prompts are

caught to policy, benign prompts are **not** over-blocked (false-positive rate within your agreed bound), groundedness flags hallucinations on the RAG app, and the incident/appeal path works.

1. Once the gate passes and trust & safety signs off, change the filter to `mode=enforce` on the

pilot app so it actually blocks/redacts per policy. Keep the observe-mode logs for comparison.

1. Confirm the rollback: setting `mode=observe` (or bypassing the filter) instantly reverts to

non-blocking. Record it in the change ticket.

Step 11 — Expand to more apps

1. Only after the pilot enforces cleanly: integrate `Safety Filter – Pre-Post Generation – Pilot` into

the next app(s) from the Step 1 worklist, each in **observe mode first**, then enforce after its own validation.

1. Reuse the same Content Safety resource (or add regional resources for latency), the same severity

policy and blocklist, and the same RAI review/red-team/incident process. Re-run `AISAFETY-Report.py` per app and on a schedule so posture does not drift.

1. Feed new jailbreaks discovered in production back into the `Red-Team Prompt Set` — red-teaming is

continuous.

Order of operations

`Assess apps → Provision resource (keyless) → Grant RBAC → Severity policy → Blocklist → Build safety filter (observe) → Integrate on pilot → RAI review → Incident/appeal process → Validate + enforce → Expand`

5. Testing Guide — Azure AI Content Safety & Responsible-AI Guardrails

Run after the safety filter is integrated on the pilot app in observe mode (Implementation Step 7), and re-run after flipping to enforce and after each expansion wave. The most important tests are the **red-team probes** (harmful/adversarial input is caught) and the **false-positive check** (benign input is *not* over-blocked). The red-team set is used to test the guardrails — the reports capture only the safety verdicts, never harmful content.

Test 1 — Guardrail configuration is correctly wired

Do: Run `AISAFETY-Validate.py`. **Expected:** The Content Safety resource is reachable **keyless** (managed identity), the `Prohibited Terms – App Guardrails – Pilot` blocklist exists, the `Content Severity Policy` config loads, and the Prompt Shields / groundedness endpoints respond (or are clearly reported as unavailable in-region so you know before you depend on them).

Test 2 — Text moderation catches harmful input to policy

Do: Run `AISAFETY-Report.py` with the `Red-Team Prompt Set – App Guardrails – Pilot`. **Expected:** Prompts designed to be hateful/violent/sexual/self-harm return severities **at or above** your policy block threshold for the right category, and the filter's decision is **block** (or **review**) per the `Content Severity Policy`.

Test 3 — Benign input is NOT over-blocked (false-positive check)

Do: Run `AISAFETY-Report.py` over the **benign control prompts** (normal, on-topic user inputs). **Expected:** Benign prompts return low severities and are **allowed**. The false-positive rate is within the bound trust & safety agreed to. If benign inputs are being blocked, raise the threshold or refine the blocklist before enforcing.

Test 4 — Custom blocklist flags its terms

Do: Submit an input containing a term from `Prohibited Terms – App Guardrails – Pilot` through the pre-filter (or `AISAFETY-Report.py`). **Expected:** The response includes a **blocklist match** for that term and the filter decision reflects your blocklist policy, independent of the category severities.

Test 5 — Prompt Shields catch jailbreak and indirect injection

Do: Submit (a) a direct jailbreak input ("ignore your instructions and…") and (b) an input where a retrieved document contains hidden instructions (indirect prompt injection), using the red-team set. **Expected:** Prompt Shields flag both as attacks and the pre-filter **blocks** them before the model is called. *If Prompt Shields is not available in your region/API version, this test is marked N/A with a note — verify current availability.*

Test 6 — Post-generation moderation catches harmful output

Do: Drive the app with an input that is itself benign but is crafted to coax harmful output; observe the post-generation filter on the model's answer. **Expected:** The answer is scored again after generation; if it exceeds policy it is **blocked/redacted** post-generation even though the input passed. (This proves you filter both sides, not just input.)

Test 7 — Groundedness detection flags hallucinations (RAG apps)

Do: On the RAG pilot app, ask a question whose true answer is **not** in the grounding sources, and pass the answer plus its sources to the groundedness check. **Expected:** The groundedness check flags the answer as **ungrounded** (low groundedness), and the filter routes it to review/block per policy. *Groundedness is a preview capability — verify availability; mark N/A with a note if unavailable.*

Test 8 — Protected-material detection (where available)

Do: Ask the app to reproduce a well-known copyrighted passage; run the output through the protected-material check. **Expected:** The check flags protected material and the filter blocks/redacts it. *Preview capability — verify availability.*

Test 9 — Observe → enforce behaves correctly

Do: With the filter in `mode=observe`, confirm a harmful input is **logged but still answered**; then set `mode=enforce` and confirm the same input is **blocked**. **Expected:** Observe logs the decision without disrupting; enforce actually blocks. Setting `observe` again is an instant rollback.

Test 10 — Incident and appeal path works

Do: Exercise the `RAI Incident & Appeal Process – App Guardrails`: report a harmful output that slipped through, and appeal a wrongly blocked benign input. **Expected:** Both route to the named owner with the logged decision attached, and the SLA/triage steps are followed.

Pass/fail gate

Gate	Requirement
Pilot pass (flip to enforce)	Tests 1–5 pass; Test 3 false-positive rate within the agreed bound; Test 6 proves post-generation filtering; the incident/appeal path (Test 10) works; trust & safety has ratified the thresholds
Expand pass	Tests 1–4, 6, and (where available) 7–8 pass for each newly integrated app **in observe mode** before that app is flipped to enforce

6. Troubleshooting Guide — Azure AI Content Safety & Responsible-AI Guardrails

Use `AISAFETY-Validate.py` to confirm the guardrail config is wired, `AISAFETY-Report.py` for the current moderation posture (red-team + benign), `AISAFETY-Discovery.py` to re-inventory apps, and `AISAFETY-Blocklist.py` (default preview) for safe blocklist changes.

A. Authentication and connectivity

Symptom	Cause	Remediation
`401`/`403` from Content Safety	Managed identity lacks the data-plane role, or wrong credential	Assign **Cognitive Services User** to the app's managed identity on the resource; confirm `DefaultAzureCredential` picks up that identity; for local testing sign in with `az login`
Works locally, fails when deployed	Local dev identity has access, the app's managed identity does not	Enable a managed identity on the app compute and grant it *Cognitive Services User* — don't rely on a key
Timeouts / connection refused	Public access disabled and no private endpoint from the app's VNet	Add a private endpoint + private DNS for the resource (global G3), or re-enable public access if that matches your architecture
App has an API key hard-coded	Anti-pattern (keys leak)	Move to keyless (managed identity); if a key is unavoidable, store it in Key Vault and reference it — never in source

B. Over-blocking (false positives)

Symptom	Cause	Remediation
Benign, on-topic inputs are blocked	Severity threshold too low for your use case	Raise the per-category block threshold in `Content Severity Policy – App Guardrails – Pilot` (re-ratified by trust & safety); re-run `AISAFETY-Report.py` on benign controls
A normal word keeps getting flagged	It is in the blocklist (or a substring match)	Review `Prohibited Terms – App Guardrails – Pilot` with `AISAFETY-Blocklist.py`; remove/refine the term
Users complain right after go-live	You flipped to enforce without observing first	Roll back to `mode=observe`, measure the false-positive rate, tune thresholds, then re-enforce

C. Under-blocking (harmful content gets through)

Symptom	Cause	Remediation
Harmful **output** slips through though input was filtered	Only the input is being moderated	Confirm the **post-generation** filter runs on the model's answer, not just the input (see Testing Test 6)

Symptom	Cause	Remediation
A jailbreak works	Prompt Shields not called, or the attack is novel	Ensure the pre-filter calls Prompt Shields on the user prompt **and** retrieved content; add the bypass to `Red-Team Prompt Set – App Guardrails – Pilot` and re-test
Category severity is high but nothing blocked	The filter isn't reading the severity policy, or is in observe mode	Confirm `mode=enforce` and that the filter compares severities to the loaded policy thresholds

D. Prompt Shields / indirect injection

• Confirm the pre-filter runs Prompt Shields over **both** the user prompt and any **documents the app retrieved** — indirect injection hides in ingested content. If Prompt Shields is unavailable in your region or API version, `AISAFETY-Validate.py` reports it; treat that capability as N/A and compensate with stricter input handling until it is available. *Verify current availability.*

E. Groundedness detection issues

Symptom	Cause	Remediation
Groundedness always "grounded" (never flags)	Sources not being passed, or the wrong sources	Pass the exact retrieved passages the answer was generated from to the groundedness call
Groundedness endpoint errors / not found	Preview capability not available in-region or API version drift	Confirm current availability and API version; design the filter to skip-with-log rather than fail the request
Everything flagged ungrounded	Answer includes reasoning beyond the sources by design	Tune what you route to review vs block; groundedness is a signal, not an absolute — keep a human in the loop

F. Protected-material detection

• Preview capability — if it errors or is unavailable, `AISAFETY-Validate.py` reports it; skip-with-log and revisit when it is GA in your region. *Verify current availability.* Do not let a preview endpoint outage fail the whole request.

G. Latency and cost

Symptom	Cause	Remediation
Noticeable added latency per request	Multiple synchronous safety round trips	Co-locate the resource with the app region; short-circuit later stages once an earlier stage already blocks; parallelize independent checks
Unexpected spend / throttling (429)	Volume above quota / no budget controls	Set budgets and alerts and request quota appropriate to volume (global G8); back off and retry on 429

H. Diagnostics quick reference

```
# Confirm the guardrail config is wired (keyless, blocklist, policy, endpoints)
python AISAFETY-Validate.py

# Read-only moderation posture: red-team + benign controls (verdicts only)
python AISAFETY-Report.py

# Re-inventory apps and current safety layer
python AISAFETY-Discovery.py

# Preview blocklist changes before applying
python AISAFETY-Blocklist.py --list
python AISAFETY-Blocklist.py --name "Prohibited Terms – App Guardrails – Pilot" --terms-file terms.txt   #
```

Portal checks: the Content Safety resource → **Access control (IAM)** (role assignments), **Networking** (private access), and **Monitoring** → **Diagnostic settings / Logs** (moderation call history and errors).

I. When to escalate

If auth is keyless and correct, the filter reads a ratified severity policy, both pre- and post-generation stages run, Prompt Shields are called on prompt and retrieved content, and harmful content **still** slips through (or benign content is still over-blocked after tuning) — capture the exact input, the safety verdict logged, and the API version in use, and open a Microsoft support case. Volatile Prompt Shields / groundedness / protected-material specifics are deferred to Microsoft's documentation. **Acceptable-content and responsible-AI policy determinations remain the customer's legal / trust-and-safety decision — this guide provides the controls, not the policy.**

7. Best Practices — Azure AI Content Safety & Responsible-AI Guardrails

- **Filter both sides, always.** Moderate the **input** (before the model) *and* the **output** (after

the model). Clean input can still produce harmful output, and harmful input must never reach the model. A one-sided filter is a false sense of safety.

- **Observe before you enforce.** Run the safety filter in `mode=observe` on the pilot first, measure the

false-positive rate on real benign traffic, then flip to enforce. Blocking users on day one with untuned thresholds destroys trust in the guardrail.

- **Thresholds are a policy decision — keep them in config.** The service returns severities; where you

block is a **trust-and-safety** call, ratified by legal/trust-and-safety and stored as configuration the filter reads. Never bury threshold numbers in code where changing policy means shipping code.

- **Stay keyless.** Authenticate the app to Content Safety with a **managed identity** and *Cognitive

Services User* (`DefaultAzureCredential`). If a key is unavoidable, it lives in **Key Vault**, never in source, logs, or config files.

- **Never log the harmful content itself.** Log the **decision** — categories, severities, shield hits,

groundedness score, and an identifier — not the raw harmful text. Your safety logs must not become a new exposure.

- **Red-team continuously, not once.** New jailbreaks and injection patterns appear constantly. Keep

`Red-Team Prompt Set – App Guardrails – Pilot` alive: add every real-world bypass you find in production back into it and re-run `AISAFETY-Report.py`.

- **Shield against indirect injection, not just direct jailbreaks.** The dangerous attacks hide

instructions inside documents, web pages, and emails your app ingests. Run retrieved content through Prompt Shields, not only the user's typed prompt.

- **Ground your answers and check the grounding.** For RAG apps, pass the retrieved sources to the

groundedness check so confident hallucinations are flagged before a user believes them.

- **Keep a human in the loop for high stakes.** Content Safety is probabilistic. For high-consequence

uses, route "review" decisions to a person rather than auto-allowing or auto-blocking.

- **Give users an appeal.** Wrongly blocked content erodes trust as much as harmful content does. The

`RAI Incident & Appeal Process – App Guardrails` must let users report both misses and false blocks, routed to a named owner.

- **Budget for the round trips.** Every pre/post call adds latency and cost. Set quotas and budgets

(global G8), consider co-locating the resource with the app, and batch/short-circuit where a stage's verdict already decides the outcome.

- **Treat preview features as preview.** Groundedness and protected-material detection evolve — verify

current availability and API versions, and design the filter to degrade gracefully (skip-with-log) if a preview endpoint is unavailable rather than failing the whole request.

Where to go next

- **Governing Microsoft 365 Copilot instead of a custom app?** That is a different problem — permissions,

sensitivity labels, and DLP over tenant data. See **Microsoft 365 Copilot Secure Adoption & Governance** and the **Microsoft Purview DLP / Sensitivity Labels / Retention** solution. This content-safety solution guardrails *custom* apps; it does not duplicate Copilot/Purview governance.

- **Building the AI app, model deployment, or RAG pipeline** the guardrails wrap — see the AI

landing-zone and RAG solutions in the AI catalog.

• **Private networking for the AI services** (private endpoints, disabled public access) — see the AI landing-zone solution and global requirement G3.

• **Identity, managed identity, and least-privilege RBAC** across your Azure estate — see the Entra ID and Azure security-baseline solutions.

8. Deliverables Summary — Azure AI Content Safety & Responsible-AI Guardrails

Documentation

Document	Purpose
Executive Summary	Business case, what's in the box, and the boundary between technical controls and policy
1. Solution Overview	How Azure AI Content Safety works (categories, severity, Prompt Shields, blocklists, groundedness, protected material) and the pre/post-generation filtering pattern
2. Requirements Guide	Content-safety-specific prerequisites with Verify / Implement-if-missing
Global Requirements (AI)	Category-wide AI prerequisites, published once
3. Implementation Guide	Exact, ordered steps: provision keyless, severity policy, blocklist, build the safety filter, integrate on a pilot in observe mode, RAI review, incident/appeal, validate + enforce, expand
5. Testing Guide	Validation procedures with red-team probes, false-positive checks, and pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations + where to go next

Python scripts

Script	What it does
`AISAFETY-Discovery.py`	Read-only: inventories in-scope AI apps/endpoints, whether a Content Safety resource exists, and whether each app calls any safety layer today

Script	What it does
`AISAFETY-Validate.py`	Read-only: confirms the guardrail config is wired — resource reachable keyless, `Prohibited Terms – App Guardrails – Pilot` blocklist present, severity policy loaded, Prompt Shields / groundedness endpoints reachable
`AISAFETY-Report.py`	Read-only: runs the `Red-Team Prompt Set – App Guardrails – Pilot` plus benign controls through the safety endpoints and reports categories/severities/shield hits and the false-positive rate (verdicts only — never emits harmful content)
`AISAFETY-SafetyFilter.py`	Integration sample: the reusable pre/post-generation `Safety Filter – Pre-Post Generation – Pilot` — moderation + blocklist + Prompt Shields pre-generation, moderation + groundedness + protected-material post-generation, decisions driven by the severity policy config; defaults to `mode=observe` (log, don't block)
`AISAFETY-Blocklist.py`	Safe blocklist manager (default preview): creates/updates `Prohibited Terms – App Guardrails – Pilot` and adds terms only with `--apply`; idempotent

*The five scripts are **Python** using the `azure-ai-contentsafety` SDK and `azure-identity` (keyless). Preview capabilities (Prompt Shields specifics, groundedness, protected-material) are called via the current API and degrade gracefully with a logged note where a capability is unavailable in-region — *verify current availability in your tenant*.*

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **Python scripts** in a single ZIP.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**
- Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

Custom AI apps with real guardrails — harmful text/image content moderated to a ratified severity policy, jailbreak and indirect-injection attacks shielded, answers checked for groundedness and protected material, and a responsible-AI review, red-team, and incident/appeal process behind it — proven on a pilot app in observe mode before any app enforces or ships.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AISAFETY-Blocklist.py

Purpose: Create/update the custom blocklist `Prohibited Terms - App Guardrails - Pilot` and add terms to it. LISTING is always read-only. Any CHANGE (create blocklist, add terms) defaults to a SAFE PREVIEW that prints exactly what it WOULD do and makes NO change; it applies only when you pass apply. Idempotent: re-running updates the same blocklist without creating duplicates.

How to run: pip install azure-ai-contentsafety azure-identity export CONTENT_SAFETY_ENDPOINT="https://<resource>.cognitiveservices.azure.com/"

AISAFETY-Discovery.py

Purpose: Assess (Step 1) which AI apps are in scope for guardrails, whether an Azure AI Content Safety resource already exists in the subscription, and whether each app calls any safety layer (pre/post-generation) today. Produces the worklist and helps pick the pilot app. READ-ONLY — it inspects an app-inventory file and lists Azure Cognitive Services accounts; it changes nothing.

How to run: pip install azure-identity azure-mgmt-cognitiveservices Optional: describe your apps in a JSON file (see --apps-file schema below). python AISAFETY-Discovery.py --subscription <sub-id> --apps-file apps.json --out discovery.json

AISAFETY-Report.py

Purpose: Run the `Red-Team Prompt Set - App Guardrails - Pilot` (adversarial prompts that TEST the guardrails) plus a set of benign control prompts through the Content Safety endpoints and report, per prompt: the category severities, any blocklist match, any Prompt Shields hit, and the resulting policy decision (allow/review/block). Summarizes the DETECTION rate on red-team prompts and the FALSE-POSITIVE rate on benign prompts — the two numbers the pilot gate needs. READ-ONLY: it evaluates prompts against the safety service and records only the SAFETY VERDICTS. It never calls a language model and never emits harmful content — only the verdict metadata.

How to run: pip install azure-ai-contentsafety azure-identity export CONTENT_SAFETY_ENDPOINT="https://<resource>.cognitiveservices.azure.com/" python AISAFETY-Report.py --policy severity_policy.json \ redteam redteam_prompts.jsonl --benign benign_prompts.jsonl --out report.json

AISAFETY-SafetyFilter.py

Purpose: The `Safety Filter - Pre-Post Generation - Pilot` integration sample. Drop this module into your AI app and call it around your model: PRE-generation : moderate the user input (+ any retrieved documents) with analyze_text (Hate/Sexual/Violence/SelfHarm severities), the custom blocklist, and Prompt Shields (jailbreak + indirect prompt-injection). POST-generation : moderate the model's answer, run a groundedness check against the RAG sources, and a protected-material check. Every decision (allow / review / block) is driven by a ratified severity policy read from config, and every decision is logged (verdicts only, never the harmful text).

AISAFETY-Validate.py

Purpose: Confirm, before you rely on the safety filter, that: the Content Safety resource is reachable KEYLESS (managed identity / DefaultAzureCredential), a benign probe returns a well-formed analyze_text result (category severities present), the custom blocklist `Prohibited Terms - App Guardrails - Pilot` exists, the severity policy config loads and has per-category thresholds, Prompt Shields / groundedness endpoints respond OR are clearly reported as unavailable in-region so you know before depending on them. READ-ONLY — sends only a benign probe string and lists config; changes nothing.

How to run: pip install azure-ai-contentsafety azure-identity export CONTENT_SAFETY_ENDPOINT="https://<resource>.cognitiveservices.azure.com/" python AISAFETY-Validate.py --policy severity_policy.json \ blocklist "Prohibited Terms - App Guardrails - Pilot"